End-of-Module Assessment (EMA) — e-Portfolio

Advanced Object-Oriented Design and Programming (AOODP)

Assignment Title: e-Portfolio — Showcase of Artefacts and Professional Development Plan

Module: Advanced Object-Oriented Design and Programming (AOODP)

Course/Programme: AOODP (Advanced Object Oriented Design & Programming)

University ID: 12701559

Submission date: 17th October 2026

e-Portfolio (GitHub repository): https://github.com/stacyalwangaessex/aoodp-capstone

e-Portfolio (published via GitHub Pages): https://stacyalwangaessex.github.io/aoodp-capstone/

Word count: 2,497 words (e-Portfolio introduction and Tasks 1–4 narrative; cover sheet, reference list, figure, code excerpts and Appendices A–C excluded)

Declaration:

I hereby certify that this is my own original work. I have used any relevant technologies according to the assignment guidelines set out on Moodle.

Signature: 12701559 Date: 17th October 2026

e-Portfolio Introduction

This e-Portfolio presents the AOODP capstone: a transaction risk-assessment engine of nine modules, 355 offline tests and 99% statement coverage. Task 1 maps six learning outcomes to code, Task 2 showcases twelve artefacts, Task 3 reflects on academic and employability skills, and Task 4 sets a development plan. Every claim is tied to a file or captured run, and each section states what the design gave up as well as what it bought.

Task 1 – Summarise Key Learning Outcomes

The capstone is a transaction risk-assessment engine: transactions in, governance-instrumented decisions out. Appendix A maps each claim to its file.

1.1 Understand and implement secure coding practices

Security is structural, not appended. Transaction is a frozen slotted dataclass whose __post_init__ calls validation.py, so an invalid transaction cannot exist, and errors never echo the rejected value, since reflecting untrusted input enables injection (OWASP Foundation, 2021). RedactingDecorator is outermost, so the audit sink never sees a raw identifier.

1.2 Apply advanced object-oriented principles to complex problems

Five concerns — features, algorithm, policy, lifecycle, format — are joined only by the composition root in service.py. Encapsulation uses frozen value objects, polymorphism RiskScoringStrategy and ReportVisitor, abstraction a five-line ScoringBackend protocol, and constructor injection: algorithm, provider, model version and format change without editing a consumer. Separation of concerns is the organising principle — one module owns each axis — and that is what makes the design scalable and extensible: a new algorithm, provider or report is added at one seam while the rest stay untouched.

1.3 Utilise design patterns for reusable, maintainable, flexible code

Pattern	Where	Flexibility it buys
Strategy	strategies.py	Scoring mode chosen by one config string
Decorator	decorators.py	Redaction, metrics, audit, caching, fallback composed once
Visitor	reporting.py	New reports as classes; existing elements untouched
Abstract Factory	providers.py	A provider family swapped together
Dependency Injection	service.py	Injected dependencies make tests deterministic

Each pattern was chosen against a named rejected alternative, and the trade-off — not the pattern's reputation — is the justification. The payoff: runtime behavioural flexibility — scoring mode, provider family and report format are configuration at the composition root, so behaviour changes without editing a consumer.

1.4 Design architectures suitable for large-scale systems, ensuring security and robustness

Caching is an inner decorator, so it can become distributed without touching governance; the registry centralises lifecycle state. Robustness is demonstrated: a provider outage produces a decision that reaches the audit trail, and a promotion that breaks a lifecycle rule is refused.

1.5 Develop software adaptable for AI models and efficient for data science tasks

The AI layer is reached only through ScoringBackend, so swapping a hosted LLM for a local one is a composition-root change. Explainability is contractual: StrategyOutput carries contributions and rationale, and FeatureDefinition is named, versioned and owned, so training and serving share definitions. The registry makes model version a runtime fact — the precondition for shadow deployment and rollback.

1.6 Link to practical work

Secure coding appears in validation.py; the four patterns in one engine (Task 2); AI adaptability in the registry and feature store, where run section 7 changes the live version without code change. Unit 6 supplies the concurrency foundation: FeatureStore takes a re-entrant lock because compute_vector calls get internally, where a plain Lock self-deadlocks.

Task 2 – Showcase Artefacts

Repository: https://github.com/stacyalwangaessex/aoodp-capstone (GitHub Pages); code cited as evidence.

2.1 Artefact A — Thread-safe concurrency (Unit 6)

bank_account.py and transaction_simulator.py, 39 tests: encapsulated state under a re-entrant lock, custom exception types, and a transfer service that cannot deadlock under nested locking.

2.2 Artefact B — Strategy pattern

RiskScoringStrategy defines score(features, model_ref), implemented rule-based, statistical and model-backed through ScoringBackend; contributions records which inputs drove the number. Strategy was chosen over an if/elif mode flag because the algorithms differ in the inputs they need and the rationale they give, not only in output, so polymorphism keeps each decision path with its explanation.

Listing 1 — the port the whole AI path depends on (capstone/strategies.py, abridged)

```
@runtime_checkable
class ScoringBackend(Protocol):
    def predict(
        self, features: Mapping[str, float], model_ref: str
    ) -> Mapping[str, Any]:
        """Return a raw prediction payload for *features*."""
        ...  # pragma: no cover - protocol declaration
```

2.3 Artefact C — Decorator pattern

Five layers — redaction, metrics, audit, caching, fallback — are composed by compose_scoring_pipeline(), the single assembly point. **Defect fixed:** assembly placed

AuditingDecorator inside FallbackDecorator; every per-decorator test passed, and only a test that a degraded decision reaches the audit sink exposed it — the challenge this pattern sets: each layer is individually correct, yet correctness lives in an ordering the type system cannot check.

Listing 2 — the ordering decision made explicit (capstone/decorators.py, abridged; indentation normalised)

```
core: ScoringComponent = StrategyComponent(strategy, policy=policy)

if cache_entries:
    core = CachingDecorator(core, max_entries=cache_entries)
if fallback_strategy is not None:
    core = FallbackDecorator(
        core, StrategyComponent(fallback_strategy, policy=policy)
    )
# Auditing wraps the fallback, not the other way round: a degraded
# decision must reach the audit trail rather than die inside the layer
# that produced it.
core = AuditingDecorator(core, sink=audit_sink)
core = MetricsDecorator(core, clock=clock)
core = RedactingDecorator(core, extra_sensitive_keys=extra_sensitive_keys)
return core
```

2.4 Artefact D — Visitor pattern

ReportVisitor declares visit_batch, visit_scored_transaction and result, with four visitors through a render(batch, visitor) driver. Reporting adds operations while elements stay immutable, so a new report is one class; a new element type would force every visitor to change, acceptable as the hierarchy is closed.

2.5 Artefact E — Abstract Factory pattern

AiProviderFactory creates a family — config, backend, explanation generator — in local heuristic and hosted LLM variants around an injected transport; assemble() returns it only after a consistency check, since a hosted endpoint with redaction off is a contradiction. No domain module imports a vendor SDK, so the remote path runs offline.

2.6 Artefacts F and G — Registry and Feature Store

ModelRegistry enforces _ALLOWED_TRANSITIONS over immutable ModelRecords; run section 7 promotes 2.2.0 to PRODUCTION while 2.1.0 is archived. **Defect fixed:** promote() returned the updated record without writing it back, so asserting on a return value hid a missing mutation. The lock-guarded store holds seven FeatureDefinitions and skips a rule whose feature is missing rather than fabricating a zero.

2.7 Artefact H — Dependency Injection

RiskAssessmentService takes every collaborator by constructor, and build_risk_service() is the composition root. ServiceContainer stays a minimal 45-line factory rather than a framework container; a service locator is kept at the outer edge because it hides dependencies from the type checker (Fowler, 2004). That is where abstraction was stopped against over-engineering: a general container would be more capable and less legible, so the smaller factory won on what a reader can verify.

2.8 Artefact I — Testing and AI-assisted tests

capstone/tests/ holds nine modules and 355 offline tests at 99% coverage of 2,512 statements, with doubles named by category — stub, spy and mock — so a reader can see which contract each isolates and every layer stays testable without a live model (Python

Software Foundation, 2026). Testing complexity here is path, not line, coverage: composition order, retry budgets and lifecycle transitions multiply, which is why doubles, not live models, carry the AI paths. An AI assistant drafted tests from class contracts, finding a boundary case I had missed — a boolean score, an int in Python, that passes isinstance — but also produced confidently wrong assertions, detectable only by running them. Both capstone defects now carry regression tests that read state back through a second access path.

Listing 3 — a spy double that makes one call reviewable (capstone/tests/conftest.py, abridged)

```
class CallRecorder:
    """A spy that records every call it receives."""

    def __init__(self) -> None:
        self.calls: list[Any] = []

    def __call__(self, *args: Any, **kwargs: Any) -> Any:
        self.calls.append((args, kwargs))
        return None

    @property
    def count(self) -> int:
        return len(self.calls)
```

2.9 Capstone — architecture and robustness

Figure 1 shows the composition root, the defended decorator order, the strategy/backend seam, adapters and visitor. evidence/demo_run_output.txt captures eight sections: a six-transaction batch (two allow, three review, one block); audit records with no identifier; masking before egress (GB33…5555 → ****…5555); and a provider outage with a three-attempt retry budget. Critically, the capture bounds the claim: rule weights are expert-set and unvalidated and the cache is in-process, so the robustness shown is single-node.

Figure 1 — Capstone architecture: composition root, scoring pipeline and extension seams

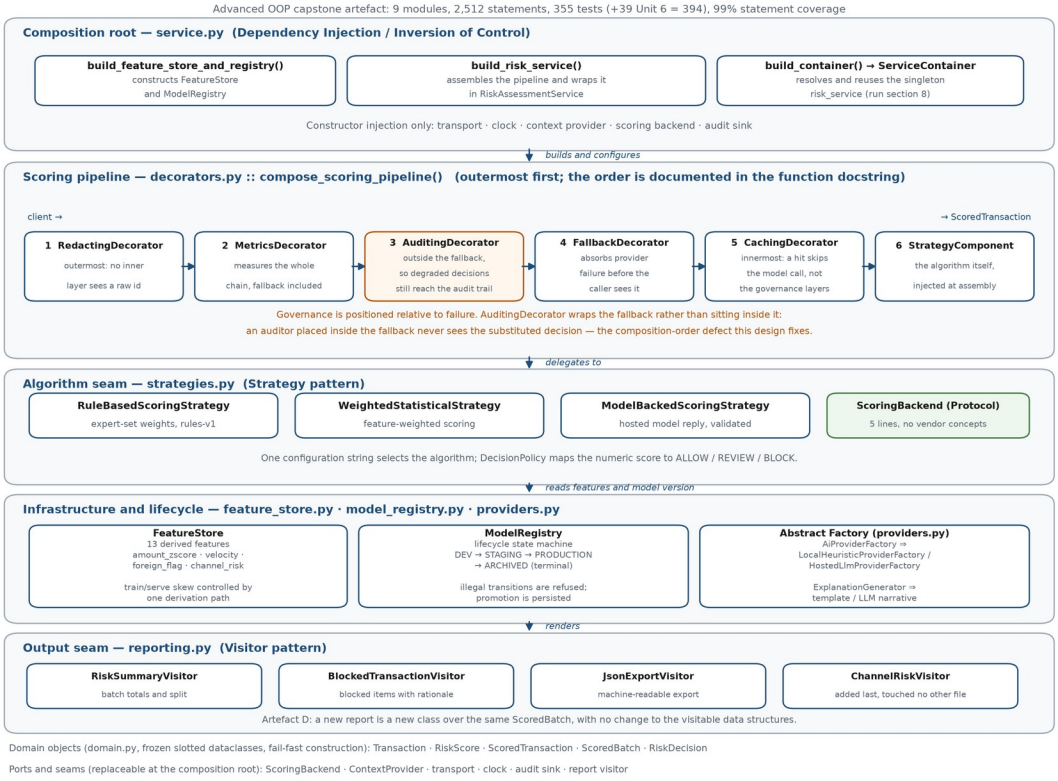


Figure 1 — Capstone architecture

Task 3 – Reflect on Skill Development

An e-Portfolio is a curated, purposeful collection of evidence rather than a chronological stack of exercises (Lorenzo and Ittelson, 2005), which is why every claim below is tied to a named artefact or capture.

3.1 Academic skills

Critical thinking. The formative experience was the composition-order defect. My first reaction was to distrust the test; the correct reaction was to distrust the design. I now ask what a passing test *cannot* see before asking whether it should pass, and applied to the registry defect that question produced my working rule about observing state through a second access path. That reframing — from "does this pass?" to "what does passing leave unexamined?" — is the transferable analytical habit, and it is what turned two green suites into evidence that the design, not the test, was wrong.

Ethical considerations in AI-driven OO design. *Bias propagation:* policy lives in data rather than a class hierarchy, so rule weights and thresholds are inspectable, versionable values a reviewer can read directly. The residual risk is real: those weights are expert-set, not learned, so they encode the blind spots of their authors — the engine makes bias inspectable, not absent (Barocas, Hardt and Narayanan, 2023). *Transparency:* `rationale` and `contributions` are contract fields, so an algorithm that cannot say which inputs drove its number cannot conform. *Black-box coupling:* `ScoringBackend` is five lines with no vendor concepts. *Auditability:* every decision produces a structured record — component, model version, score, decision, rationale — asserted by the run to contain no account or customer identifier.

Problem-solving and refactoring. Refactoring was driven by measurable signals, not taste: extracting `validation.py` removed duplication across five modules, and extracting `compose_scoring_pipeline()` collapsed four copies of an ordering decision into one documented function. I also had to manage abstraction against over-engineering: a shared base class for the reporting visitors would create inheritance without polymorphism, so the duplication stayed.

Research and self-study. Reading fell into three groups: pattern sources (Gamma *et al.*, 1994; Martin, 2000, 2017; Fowler, 2004, 2018) for vocabulary and trade-offs; AI-systems sources (Sculley *et al.*, 2015) for the feature-store and registry arguments; and risk and fairness sources (Aven and Thekdi, 2025; Barocas, Hardt and Narayanan, 2023). The useful outcome was negative: none of the pattern literature treats governance layers — audit, redaction, degradation — as first-class composition concerns, so the pipeline-order reasoning here came from analysing my own failure modes.

3.2 Employability skills

Time management. I worked in artefact-sized increments, capturing evidence alongside construction rather than reconstructing it at the end. My concrete failure was underestimating test-writing: closing coverage from 87% to 99% took three modules and 143 tests and produced the two most valuable findings in the portfolio, which argues for budgeting test time as design time. The same lesson held for this portfolio: the narrative was rewritten three times as the evidence arrived, which cost more than capturing it while the work was fresh.

Communication and literacy. The clearest gain was writing docstrings that carry a decision rather than a description: `compose_scoring_pipeline` states the order and justifies each position. Writing the degradation marker as a machine-readable prefix (`[degraded: primary unavailable]`)

was also a communication decision, since the same form is legible to an operator, a downstream consumer and a test.

Resilience and interpersonal skills. Resilience meant sitting with a failing test I believed was correct for longer than was comfortable, and changing the design instead of the assertion.

Collaboration and independent working. This is individual work with a single-author history and no peer review; Appendix B records that as a gap rather than substituting retrospective minutes. Collaboration was instead adversarial and tool-mediated: an AI assistant drafted tests against my contracts and, where its assertions were confidently wrong, I overruled it. That draft is not peer review, but it exercised the discipline peer review demands: writing each decision so another party can audit it. Stating the decorator-ordering decision that way forced me to justify an intuition I had only relied on.

Task 4 – Professional Development Plan (PDP)

4.1 Strengths and areas for improvement

Strengths (evidenced)	Areas for improvement (evidenced)
Nine modules, clear boundaries, one composition root	Intent documented unevenly: reasoning in docstrings, not one design record
Test discipline as design: 355 tests, 99% coverage	Test-*shape* thinking arrived late; two defects escaped
Security as structure: immutable objects, boundary redaction	Fairness metrics and drift understood, not implemented
Registry and feature store implemented, not named	Retries, fallback and audit exist; no monitoring artefact
Honest scoping: abstractions rejected, risks documented	Early drafts descriptive, not reasoning

4.2 Future goals and actions

Goal 1 — AI-aware software architecture. *Certification:* AWS Certified Machine Learning Engineer – Associate, which forces the operational vocabulary — monitoring, endpoint scaling, drift — the design assumes. *Reading:* Sculley *et al.* (2015), Kleppmann (2017) on distributed-systems reasoning, and the MLflow (MLflow, 2026) and Feast (Feast, 2026) documentation. *Applied project:* a persistent registry with an append-only promotion audit and drift detection as a new visitor.

Goal 2 — Advanced testing strategies for intelligent systems. *Reading and practice:* property-based testing with Hypothesis (Hypothesis, 2026), because missing state mutation and wrong composition order are exactly what it targets — a property equating the registry's production version with the last promoted one would have caught it. *Skill:* contract testing for `ScoringBackend`. *Applied project:* an evaluation harness comparing `rules-v1` with a trained model on held-out data.

Goal 3 — Ethical software engineering in AI-integrated applications. *Reading:* Barocas, Hardt and Narayanan (2023) properly rather than selectively; the NIST AI Risk Management Framework (NIST, 2023) as governance structure; and the EU AI Act's high-risk obligations (European Parliament and Council, 2024), since credit scoring is a named high-risk category. *Practice:* model cards (Mitchell *et al.*, 2019) as required artefacts for every

registered version. *Applied project:* the fairness evaluation as a fifth reporting visitor, making bias a repeated measurement.

4.3 Reflective model — Gibbs' Cycle

Gibbs (1988) frames the event that produced the most learning: the composition-order defect. Kolb's (1984) cycle, the Johari Window (Luft and Ingham, 1955) and Bloom's revised taxonomy (Anderson and Krathwohl, 2001) were the alternatives; Gibbs was chosen because the trigger is one bounded failure whose cause can be traced.

1. Description. I had assembled the pipeline and written a test module per decorator; all passed. I then wrote a test asserting that when the hosted provider is unreachable, the degraded decision still reaches the audit trail. It failed: the sink recorded nothing for that transaction.

2. Feelings. Frustration, and a suspicion the new test was wrong, because the per-decorator tests were green; then unease on inspecting the assembly code, where the ordering was plainly visible. The discomfort marked the moment my mental model shifted from "a chain of layers" to "a chain of layers *with governance positioned relative to failure*".

3. Evaluation. The immediate cause was that AuditingDecorator wrapped the strategy before FallbackDecorator wrapped the result, so the substituted decision was produced outside the auditor. The deeper cause was that four places assembled the pipeline, making the ordering an implicit convention nobody could review; without the injectable sink it would have surfaced in production.

4. Analysis. The Decorator pattern's strength — composable, independent layers — is also its hazard: correctness depends on ordering, and ordering is invisible to the type system because every decorator shares one type. Consolidating assembly into one documented function converts a hidden convention into a reviewable artefact. Mitchell *et al.* (2019) argue a model's intended use should be a first-class artefact; the equivalent here is that each layer's *position relative to failure* deserves the same explicitness.

5. Conclusion. Individually correct tests can sum to a collectively incorrect system, and that is a design smell rather than a testing gap. Two rules followed: governance layers are positioned relative to failure modes, with the positioning documented at the assembly point, and every stateful component has at least one test reading state back through a different path.

6. Action plan. Next project I will (i) write the assembly function before the individual layers, so the ordering is decided once and deliberately; (ii) write one integration test per *failure mode* rather than per class; and (iii) re-read every assembly point at review asking "what does this ordering make invisible?".

References

Anderson, L.W. and Krathwohl, D.R. (2001) *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman.

Aven, T. and Thekdi, S. (2025) *Risk Science: An Introduction*. 2nd edn. Abingdon: Routledge.

Barocas, S., Hardt, M. and Narayanan, A. (2023) *Fairness and Machine Learning: Limitations and Opportunities*. Cambridge, MA: MIT Press.

European Parliament and Council (2024) Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L 1689. Available at: https://eur-lex.europa.eu/eli/reg/2024/1689/oj (Accessed: 17 October 2026).

Feast (2026) *Feast: Feature Store for Machine Learning*. Available at: https://docs.feast.dev/ (Accessed: 17 October 2026).

Fowler, M. (2004) *Inversion of Control Containers and the Dependency Injection Pattern*. Available at: https://martinfowler.com/articles/injection.html (Accessed: 17 October 2026).

Fowler, M. (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston, MA: Addison-Wesley.

Gamma, E. *et al.* (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley.

Gibbs, G. (1988) *Learning by Doing: A Guide to Teaching and Learning Methods*. Oxford: Further Education Unit, Oxford Polytechnic.

Hypothesis (2026) *Hypothesis: Property-Based Testing for Python*. Available at: https://hypothesis.readthedocs.io/ (Accessed: 17 October 2026).

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Kolb, D.A. (1984) *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice Hall.

Lorenzo, G. and Ittelson, J. (2005) *An Overview of E-Portfolios*. Boulder, CO: Educause Learning Initiative.

Luft, J. and Ingham, H. (1955) 'The Johari window: a graphic model of interpersonal awareness', *Proceedings of the Western Training Laboratory in Group Development*. Los Angeles: University of California.

Martin, R.C. (2000) *Design Principles and Design Patterns*. Available at: https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf (Accessed: 17 October 2026).

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall.

Mitchell, M. *et al.* (2019) 'Model cards for model reporting', *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, pp. 220–229. doi:10.1145/3287560.3287596.

MLflow (2026) *MLflow Model Registry*. Available at: https://mlflow.org/docs/latest/model-registry.html (Accessed: 17 October 2026).

National Institute of Standards and Technology (NIST) (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. Gaithersburg, MD: NIST.

OWASP Foundation (2021) *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*. Available at: https://owasp.org/Top10/ (Accessed: 17 October 2026).

Python Software Foundation (2026) *unittest.mock — mock object library*. Available at: https://docs.python.org/3/library/unittest.mock.html (Accessed: 17 October 2026).

Sculley, D. *et al.* (2015) 'Hidden technical debt in machine learning systems', *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pp. 2503–2511.

Appendix A — Artefact and evidence index

#	Artefact	File(s)	Evidence
A	Thread-safe exercise (Unit 6)	unit6/bank_account.py, unit6/transaction_simulator.py	39 tests; simulator run
B	Strategy	capstone/strategies.py	test_strategies.py; run §2
C	Decorator	capstone/decorators.py	test_decorators.py; run §6a
D	Visitor	capstone/reporting.py	test_reporting.py; run §3
E	Abstract Factory	capstone/providers.py	test_providers.py; run §6b–6c
F	Model Registry	capstone/model_registry.py	test_model_registry.py; run §7
G	Feature Store	capstone/feature_store.py	test_feature_store.py; run §2
H	DI / IoC	capstone/service.py	test_service.py; run §8
I	Testing / AI-assisted	capstone/tests/ (nine modules)	test_run_output.txt, coverage_report.txt
J	Secure validation	capstone/validation.py, domain.py	test_validation.py; run §5
K	Runtime demonstration	demo_run.py	demo_run_output.txt (8 sections)
L	Coverage report	—	evidence/coverage_html/index.html

Appendix B — Evidence held in support of independent working

Evidence type	Record that would evidence it	Held
Repository contribution history	git log, git shortlog -sne	Held — summarised in docs/git_contribution_history.txt
Task ownership	Trello/Jira/GitHub Projects board	Not held
Meeting records	Dated minutes or notes	Not held
Peer review	Comments on another's artefact	Not held
Tutor feedback	Feedback on drafts, with the response	Not held
Independent working	Dated study log or planner	Not held

Appendix C — Honest statement of limits

1. **Rule weights are expert judgement, not evidence.** They are versioned and inspectable, but inspectable is not validated, and the fairness risk in Task 3 lives here.

2. **The model-backed path runs against test doubles, never a real model.** That validates the integration contract, not that a real model answers correctly; the hosted transport is the seam where that changes.

3. **The registry and feature store are in-memory.** Lifecycle and versioning semantics are demonstrated, but durability, migration and multi-process consistency are out of scope — hence Goal 1's persisted registry.